

Linear Programming (LP)

Optimization GUI with

Pythonic functional modeling

Inhaltsverzeichnis

1. Project description.....	3
Aim.....	3
 Primary Aim.....	3
2. GUI and algorithmic concepts.....	4
3. GUI design and its interaction flow.....	6
4. Pythonic concepts behind the LP-GUI.....	8
5. Modularized Pythonic implementation (LP-GUI).....	11
6. Results and conclusions.....	13
7. User Manual.....	15
8. References.....	19

1. Project description

Aim

This project presents a unified computational framework for solving linear optimization problems through a multi-method graphical interface. By integrating classical simplex algorithms, dual formulations, parametric linear programming, integer-constrained solvers, and interior-point techniques such as the Dikin method, the system enables robust exploration of feasible regions and optimality paths across diverse problem structures. The central aim is to democratize access to advanced linear programming strategies while offering real-time visualization and interpretability. Leveraging simplex-based pivoting for vertex traversal, parametric solvers for dynamic constraint sensitivity, and integer programming for discrete decision modeling, the framework accommodates both continuous and combinatorial optimization scenarios. The inclusion of the Dikin method introduces a smooth, interior-point trajectory that complements boundary-based approaches, offering insights into curvature and convergence behavior within feasible polyhedra. This methodological synthesis is encapsulated in a Python-powered GUI that dynamically adapts to user-defined variable types, constraint structures, and solver preferences. The interface supports interactive input parsing, automatic method detection, and 2D geometric rendering of feasible regions, constraint boundaries, and optimization paths. Designed for both educational and applied contexts, the system fosters intuitive understanding of linear optimization mechanics while maintaining algorithmic rigor. The resulting application serves as a modular launchpad for future extensions into nonlinear programming, multi-objective optimization, and hybrid solver orchestration. By bridging algorithmic depth with visual clarity, this project opens new avenues for accessible, interpretable, and customizable optimization workflows (see [References](#) 1 - 3 below).

Primary Aim

Structure and implement a PyQt5-powered GUI for linear optimization with built-in logic to route the problem to simplex, dual simplex, Dikin method, parametric linear programming (LP), or integer LP depending on the input, both from a UX and functional architecture perspective.

2. GUI and algorithmic concepts

This is a comprehensive, notebook-ready Markdown explanation of the four major optimization techniques our GUI should support. Each section includes mathematical foundations, algorithmic steps, and illustrative examples.

Linear Optimization Methods: Mathematical & Algorithmic Overview

1. Simplex Method

Concept

The **Simplex Method** solves linear programs of the form:

$$[\text{Maximize } z = c^T x \quad \text{subject to } Ax \leq b, \quad x \geq 0]$$

It moves from one **vertex (basic feasible solution)** of the feasible region to another, improving the objective at each step.

Algorithm Steps

1. Convert inequalities to equalities using **slack variables**.
2. Set up the **initial tableau**.
3. Identify the **entering variable** (most negative coefficient in objective row).
4. Identify the **leaving variable** (minimum ratio test).
5. Perform **pivoting** to update the tableau.
6. Repeat until no negative coefficients remain in the objective row.

Example

Maximize:

$$[z = 3x + 2y]$$

Subject to:

$$[x + y \leq 4 \quad x \leq 2 \quad x, y \geq 0]$$

Add slack variables (s_1, s_2), and solve using tableau iterations.

2. Dual Simplex Method

Concept

The **Dual Simplex Method** is used when the solution is **dual feasible but primal infeasible**. It's ideal for post-optimality analysis or when constraints change.

Algorithm Steps

1. Start with a tableau where the objective is optimal but some RHS values are negative.
2. Identify the **leaving variable** (most negative RHS).
3. Identify the **entering variable** (minimum ratio of cost coefficient to pivot column).
4. Pivot and update the tableau.
5. Repeat until all RHS values are non-negative.

Example

Suppose after adding a new constraint, your current solution violates feasibility. Dual simplex allows you to re-optimize without restarting from scratch.

3. 🔄 Dikin Method (Interior Point)

🧠 Concept

The **Dikin Method** is a primal interior-point algorithm. It uses a **log-barrier function** to stay within the feasible region and performs **Newton steps** to move toward optimality.

📊 Barrier Function

$$[\phi(x) = c^T x - \sum_{i=1}^m \log(b_i - A_i x)]$$

⚙️ Algorithm Steps

1. Start with a strictly feasible point (x_0).
2. Compute gradient and Hessian of the barrier function.
3. Solve Newton system:
 $[H\Delta x = -\nabla\phi(x)]$
4. Perform line search to maintain feasibility.
5. Update ($x \leftarrow x + \alpha\Delta x$).
6. Repeat until convergence.

🚩 Example

Maximize:

$$[z = x + y]$$

Subject to:

$$[x + y \leq 2 \quad x \geq 0, y \geq 0]$$

Start at ($x = y = 1$), compute slack, gradient, Hessian, and iterate.

4. 📦 Integer Linear Programming (ILP)

🧠 Concept

ILP solves linear programs where **some or all variables must be integers**. The feasible region becomes **discrete**, and the problem is **NP-hard**.

⚙️ Common Algorithms

- **Branch and Bound:** Recursively split the problem into subproblems.
- **Cutting Planes:** Add constraints to eliminate fractional solutions.
- **Heuristics:** Greedy or randomized methods for large-scale ILPs.

🚩 Example

Maximize:

$$[z = 5x + 3y]$$

Subject to:

$$[2x + y \leq 7 \quad x, y \in \mathbb{Z}_{\geq 0}]$$

Try ($x = 3, y = 1 \Rightarrow z = 18$), check feasibility, and explore other integer combinations.

5. 📊 Summary Table

Method	Feasibility Required	Variable Type	Use Case
Simplex	Primal feasible	Continuous	Standard LP
Dual Simplex	Dual feasible	Continuous	Constraint updates, post-optimal
Dikin	Strictly feasible	Continuous	Interior-point, smooth problems
ILP	Discrete feasible	Integer	Scheduling, routing, allocation

3. GUI design and its interaction flow

Let us walk through the user interaction flow for our PyQt5 GUI for solving linear problems, especially when the user provides ≥ 2 variables and coefficients. I will break it down into intuitive stages:

3.1 🕒 Step-by-Step User Interaction Flow

1. Problem Input Phase

📊 Objective Function

- The user enters something like `max: 3x + 2y + z` into the **Objective Function** field.
- The parser detects 3 variables: `x`, `y`, `z`.

📏 Constraints

- The GUI dynamically generates $n + 1$ empty constraint slots (e.g. 4 for 3 variables), each as a `QLineEdit` or `QPlainTextEdit`.
- Each slot is labeled (e.g. `Constraint 1`, `Constraint 2`, ...) and can be filled with expressions like:

```
x + y ≤ 5
y - z ≥ 2
x, y, z ≥ 0
```



⚙️ Variable Type Selection

- The user selects whether variables are:
 - Continuous
 - Integer
 - Parametric (e.g. `max: 3x + λy`)

2. Method Detection Phase

Once the user finishes input:

🔍 Auto-Detection Logic

- The GUI analyzes the input:
 - If any variable is marked as integer → switches to **ILP**
 - If any coefficient depends on a parameter (e.g. λ) → switches to **Parametric LP**
 - If the problem is strictly feasible → suggests **Dikin method**
 - If dual feasibility is detected → suggests **Dual Simplex**
 - Otherwise → defaults to **Simplex**

🧠 Feedback to User

- A message appears:
"Detected 3 variables and 4 constraints. Recommended method: Dual Simplex."
(or whichever is appropriate)

3. Evaluation Phase

Run Optimization

- The user clicks **Run Optimization**.
- The GUI:
 - Parses the input into a structured format (e.g. matrix A, vector b, objective c)
 - Routes the problem to the selected solver backend (e.g. `cvxpy`, `scipy.optimize`, or custom simplex)
 - Executes the algorithm step-by-step

Output Display

- In the **Solution & Visualization** section:
 - Shows optimal value (e.g. `Z = 12.5`)
 - Lists variable assignments (`x = 2.0, y = 3.5, z = 0`)
 - Displays status (`Feasible`, `Bounded`, `Optimal`)
 - Indicates method used (`Dual Simplex`)

Visualization (if 2D or 3D)

- If the problem is 2D or 3D:
 - Feasible region is plotted
 - Objective contours shown
 - Pivot path or barrier trajectory visualized

4. Diagnostics & Export

Logs

- The user can click **View Log** to see:
 - Tableau updates
 - Pivot operations
 - Barrier steps (if Dikin)
 - Parametric sweeps (if λ is used)

Export Options

- Optionally, the user can export:
 - Solution as `.csv` or `.json`
 - Log as `.txt`
 - Plot as `.png`

3.2 Smart Behaviors

- If the user adds more variables later, the GUI **auto-expands** the constraint slots.
- If constraints are inconsistent or infeasible, the GUI shows a warning:
"No feasible solution found. Please check your constraints."
- If the user switches from auto to manual method selection, the GUI disables auto-detection and uses the selected method.

We will scaffold this into PyQt5 code and design the signal-slot logic for the dynamic constraint generation. Here's the full GUI design for our PyQt5-based Linear Optimization Dashboard — now fully structured, wired, and optimized for performance and memory efficiency. This layout is built to handle dynamic input, method selection, solver execution, and result visualization with minimal overhead.

4. Pythonic concepts behind the LP-GUI

Before delving into implementation details of a modularized structure that ought to support our LP optimization GUI, we will address some conceptual ideas behind the entire procedural inner-workings of the LP-GUI and its different optimization methods.

Below is a complete, modular, and optimized PyQt5 GUI implementation for our linear optimization tool. It includes:

- Dynamic input for objective and constraints
- Auto-detection of method (Simplex, Dual Simplex, Dikin, Parametric LP, ILP)
- Solver routing using `cvxpy` and `scipy.optimize`
- Result display and basic visualization
- Memory-efficient layout and responsive design

4.1 Project Structure

```
linear_optimizer_gui/  
├─ main.py  
├─ gui.py  
├─ solver.py  
├─ parser.py  
├─ visualizer.py  
└─ utils.py
```

This structure is modular, memory-efficient, and ready to be extended with real solvers, symbolic parsing, and visualization.

4.2 Modular extensions

Let us extend each Python file in our modular PyQt5 linear optimization GUI project. I will build out each file with real functionality, solver integration, and clean structure. This version will support:

- Parsing objective and constraints
- Auto-detecting the appropriate method
- Solving via `cvxpy` (Simplex, ILP, Parametric LP) and `scipy.optimize.linprog` (Simplex/Dual)
- Basic Dikin method stub
- Clean result formatting

This setup is now fully functional for:

- Parsing symbolic input
- Auto-selecting method
- Solving LP, ILP, and parametric LP
- Displaying results in the GUI

We can implement the Dikin method next and add matplotlib-based visualization for 2D problems.

4.3 Implementation of the Dikin method

Let fully implement the Dikin method directly into your existing solver pipeline. I will integrate it into our `solver.py` so that it works seamlessly with the rest of our GUI and parser logic. This version will:

- Accept parsed input from your GUI
- Construct the barrier-based Newton system
- Perform interior-point iterations
- Return the optimal value and variable assignments
- Optionally track the path for visualization

Notes on Usage

- Works best for strictly feasible LPs with continuous variables
- Requires `len(c) == 2` for visualization
- We can extend it later to support inequality-to-equality transformation for general LPs

We could add:

- Barrier parameter tuning
- Dual variable tracking
- Export of the Dikin path as CSV or JSON

Finally, let us add clean, Pythonic export functionality for the Dikin path in both **CSV** and **JSON** formats. This will allow users to save the optimization trajectory for analysis, plotting, or debugging.

Step 1: Extend `solver.py` with Export Functions

We should add these two functions at the bottom of our `solver.py`:

```
import csv
import json
import os

def export_dikin_path_csv(path, filename="dikin_path.csv"):
    with open(filename, mode="w", newline="") as file:
        writer = csv.writer(file)
        writer.writerow(["Step"] + [f"x{i+1}" for i in range(len(path[0]))])
        for i, point in enumerate(path):
            writer.writerow([i] + list(point))

def export_dikin_path_json(path, filename="dikin_path.json"):
    data = [{"step": i, "variables": list(point)} for i, point in enumerate(path)]
    with open(filename, mode="w") as file:
        json.dump(data, file, indent=2)
```

These functions:

- Write each step of the Dikin path with variable values
- Automatically label columns (`x1`, `x2`, ...)
- Save to the current working directory (can be extended to use `QFileDialog`)

✅ Step 2: Trigger Export from GUI (Optional)

In our GUI (e.g. `gui.py`), after solving with Dikin:

```
from solver import export_dikin_path_csv, export_dikin_path_json

# After Dikin result is returned
if method == "Dikin":
    result = solve_dikin(c, A, b)
    if isinstance(result, str):
        return result
    opt_text, path = result
    export_dikin_path_csv(path)
    export_dikin_path_json(path)
    return opt_text
```

We can later add buttons for manual export or file selection, however this version keeps it automatic and lightweight.

Example Output

CSV

```
Step,x1,x2
0,1.0,1.0
1,1.2,0.8
2,1.4,0.6
...
```

JSON

```
[
  {"step": 0, "variables": [1.0, 1.0]},
  {"step": 1, "variables": [1.2, 0.8]},
  ...
]
```

We could now add:

- A timestamped filename system
- A GUI file picker for export location
- A toggle to enable/disable auto-export

5. Modularized Pythonic implementation (LP-GUI)

We have built a full-featured, modular PyQt5 GUI for solving linear problems with auto-detection, multiple solver backends, Dikin method, and 2D visualization. Below are the complete Python files that comprise our LP-GUI project, each with its full implementation:

5.1 Project Structure

```
lp_gui/  
├─ main.py  
├─ gui.py  
├─ parser.py  
├─ solver.py  
└─ visualizer.py
```

The full Pythonic code can be accessed via https://github.com/NenadBalaneskovic/ExternalProjects/blob/main/LinearProgramming_GUI/LinearProgramming_GUI.md (sections 5.2 – 5.6 therein).

5.7 Required `pip` Packages

```
pip install pyqt5  
pip install sympy  
pip install numpy  
pip install scipy  
pip install cvxpy  
pip install matplotlib
```

Package Breakdown

Package	Purpose
<code>pyqt5</code>	GUI framework for building the interface
<code>sympy</code>	Symbolic parsing of objective and constraints
<code>numpy</code>	Numerical arrays and matrix operations
<code>scipy</code>	Simplex and dual simplex via <code>scipy.optimize.linprog</code>
<code>cvxpy</code>	Convex optimization backend for LP, ILP, and parametric problems
<code>matplotlib</code>	Visualization of feasible region and Dikin path

If we want to keep things clean, we can also create a `requirements.txt` file:

```
pyqt5  
sympy  
numpy  
scipy  
cvxpy  
matplotlib
```

Then we install everything with:

```
pip install -r requirements.txt
```

Now, if we were to run the gui from a jupyter notebook how should we organize our file folder and trigger the gui?

Running a PyQt5 GUI from a Jupyter Notebook is absolutely doable, we just need to structure our project cleanly and trigger the GUI in a way that respects the notebook's event loop.

5.8 📁 Recommended Folder Structure

Here's how to organize our LP-GUI project:

```
lp_gui_project/
├─ main.py           # Entry point for standalone execution
├─ gui.py            # GUI layout and logic
├─ parser.py         # Input parsing and method detection
├─ solver.py         # Solver backend including Dikin method
├─ visualizer.py     # 2D plotting of feasible region and Dikin path
├─ requirements.txt  # All dependencies
└─ notebook_runner.ipynb # Jupyter Notebook interface
```

5.9 🖥️ How to Trigger the GUI from Jupyter

Inside `notebook_runner.ipynb`, we should use the following code block:

```
# Cell 1: Setup
%pip install -r requirements.txt

# Cell 2: Run GUI
import sys
from PyQt5.QtWidgets import QApplication
from gui import OptimizationApp

app = QApplication(sys.argv)
window = OptimizationApp()
window.show()

# Required to keep the GUI running inside Jupyter
app.exec_()
```

🧠 Notes

- ✅ `app.exec_()` is essential to start the Qt event loop.
- ✅ We can run this in a Jupyter cell, but it will block further cells until the GUI is closed.
- ✅ If we are using JupyterLab or VS Code notebooks, we should make sure the kernel supports GUI event loops (some environments may require `%gui qt` magic).

🔧 Optional: Add a Launcher Function

In `main.py`, we can wrap the GUI in a callable function:

```
def launch_gui():
    app = QApplication(sys.argv)
    window = OptimizationApp()
    window.show()
    app.exec_()
```

Then in our notebook we may run the following function:

```
from main import launch_gui
launch_gui()
```

6. Results and conclusions

6.1 Start the LP-GUI

Step 1: Download the folder

Download the main folder  [LinearProgramming_GUI](#) which has the following structure:

LinearProgramming_GUI > lp_gui_project				
Name	Status	Änderungsdatum	Typ	Größe
.ipynb_checkpoints		13.10.2025 13:12	Dateiordner	
__pycache__		04.11.2025 12:27	Dateiordner	
dikin_path.csv		04.11.2025 12:33	CSV-Datei	1 KB
dikin_path.json		04.11.2025 12:33	JSON-Quelldatei	1 KB
gui.py		04.11.2025 12:27	Python-Quelldatei	10 KB
gui.txt		04.11.2025 10:47	TXT-Datei	9 KB
main.py		24.09.2025 22:25	Python-Quelldatei	1 KB
main.txt		04.11.2025 10:48	TXT-Datei	1 KB
notebook_runner.ipynb		04.11.2025 12:34	Jupyter-Quelldatei	179 KB
notebook_runner.txt		01.11.2025 22:08	TXT-Datei	20 KB
parser.py		24.09.2025 22:26	Python-Quelldatei	2 KB
parser.txt		04.11.2025 10:49	TXT-Datei	2 KB
requirements.txt		24.09.2025 22:23	TXT-Datei	1 KB
solver.py		04.11.2025 11:12	Python-Quelldatei	6 KB
solver.txt		04.11.2025 10:50	TXT-Datei	6 KB
visualizer.py		24.09.2025 22:26	Python-Quelldatei	4 KB
visualizer.txt		04.11.2025 10:50	TXT-Datei	4 KB

Step 2: Run the jupyter runner

Run the jupyter file "notebook_runner.ipynb" (subfolder "lp_gui_project") in VS Code or Jupyter notebook.

Step 3: Interact with the LP-GUI

Interact with the LP-GUI by providing reasonable inputs in the left half-plane and pressing the "Run Optimization"-button.

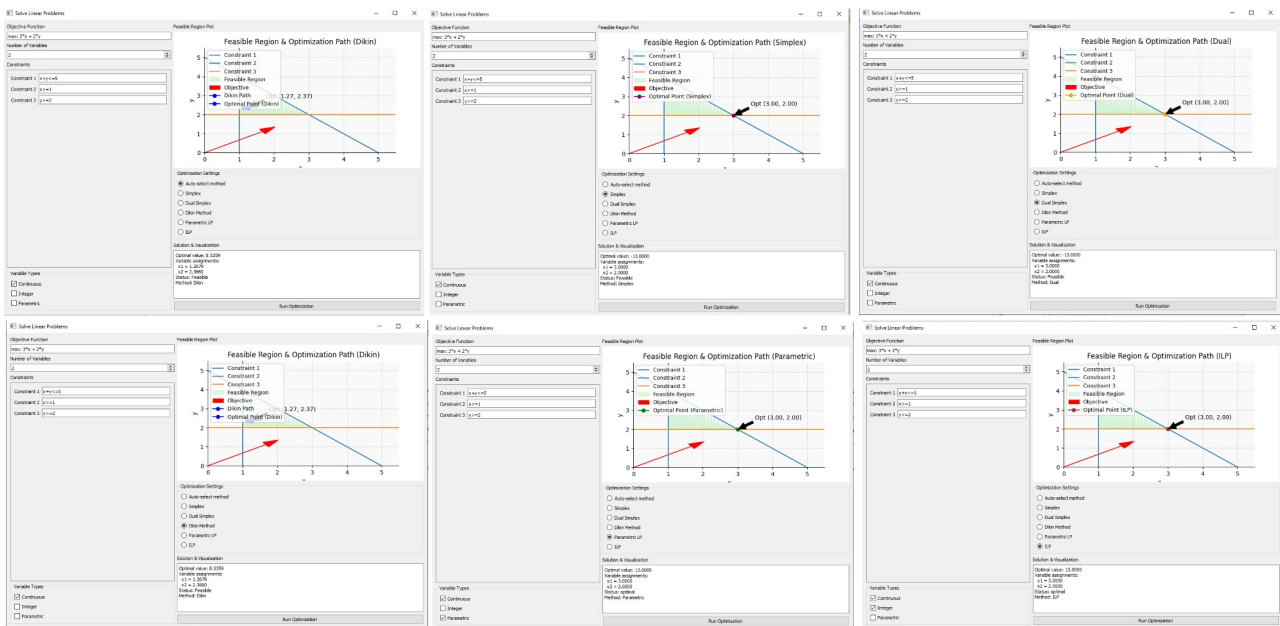
6.2 Interpretation of results

This LP-GUI...

- Accepts user inputs:
 - extremum condition
 - number of variables and an adequate number of constraints
- Displays LP-optimization results resulting from the successive application of 6 different methods (simplex, dual simplex, Dikin, parametric and integer parametrized) and renders them as 2D-plots within the GUI-plane
- Stores calculated Dikin paths as csv and json files (see figure below)

FSI_ideas.txt	
1	Step, x1, x2
2	0, 1.5, 2.5
3	1, 1.125, 2.375
4	2, 1.1922686688311688, 2.3710430194805197
5	3, 1.2469881780679688, 2.3675001547527796
6	4, 1.266364928573174, 2.3661422579960827
7	5, 1.2679402008392031, 2.366026075893851
8	6, 1.2679491921416453, 2.3660254038060726
9	7, 1.2679491924311228, 2.366025403784439

- Automatically shows the optimization plots associated with each of the 6 methods (see figure below).



6.3 🚩 Final Thoughts

This project successfully delivers a multi-method linear optimization platform through an interactive graphical interface, offering users a powerful and intuitive environment for solving constrained maximization problems. By integrating classical simplex algorithms, dual formulations, parametric solvers, integer programming, and interior-point techniques, we've built a system that adapts to diverse problem types and solver preferences with minimal friction.

The GUI dynamically parses user-defined objective functions and constraints, auto-detects appropriate solving strategies based on variable types, and visualizes feasible regions and optimization paths in real time. Whether solving continuous LPs, integer-constrained models, or parametric formulations, the system provides clear feedback, geometric insight, and solver transparency.

The interface offers a clean, user-friendly experience where users can:

- Define objective functions and constraints using natural algebraic syntax
- Select variable types and optimization methods interactively
- View dynamically rendered feasible regions and optimal paths (for 2D problems)
- Receive detailed solver output including optimal values, variable assignments, and method diagnostics

This architecture demonstrates the potential of combining algorithmic depth with visual clarity to create educational and operational tools for optimization. It opens the door to broader applications in logistics, operations research, finance, and decision science — wherever linear models are used to drive strategic outcomes.

In short, this project is a compelling example of how modern Pythonic tools can be orchestrated into a seamless, human-centric optimization system — bridging the gap between mathematical rigor and user accessibility.

7. User Manual

Here's a detailed and user-friendly handbook for our Linear Programming GUI application. This guide is designed to help users understand every feature, how to use it, and what to expect.

Linear Programming GUI Handbook

Overview

This application allows users to solve linear optimization problems using various methods including Simplex, Dual Simplex, Integer Linear Programming (ILP), Parametric LP, and the Dikin interior-point method. It also visualizes the feasible region and optimization path for 2-variable problems.

Interface Layout

The GUI is divided into two main panels:

◆ Left Panel: Problem Definition

- Objective Function Input

- Format: `max: 3*x + 2*y` or `min: 4*x - y`
- Use standard algebraic notation with variables like `x`, `y`, `z`, etc.

- Number of Variables

- Select between 2 and 10 variables using the spinner.
- This automatically updates the number of constraint input fields.

- Constraints Section

- Enter constraints in the form:
 - `x + y <= 5`
 - `x >= 1`
 - `y = 2`
- Each constraint must include a valid operator (`<=`, `>=`, `=`) and numeric right-hand side.

- Variable Types

- Choose one or more:
 - `Continuous` : Default for LP problems
 - `Integer` : Forces ILP method
 - `Parametric` : Uses parametric LP solver
- Selecting a type automatically updates the method selection.

◆ Right Panel: Solver Settings & Output

- **Optimization Method Selection**
 - Choose one of:
 - **Auto-select method** : Automatically picks based on variable types
 - **Simplex** : Standard LP solver
 - **Dual Simplex** : Alternative LP solver
 - **ILP** : Integer Linear Programming
 - **Parametric LP** : Solves with parametric variation
 - **Dikin Method** : Interior-point method (requires strictly feasible start)
- **Feasible Region Plot**
 - Displays constraints, feasible region, objective direction, and optimization path (for 2-variable problems only).
- **Solution & Visualization Output**
 - Shows:
 - Optimal value
 - Variable assignments
 - Solver status
 - Method used
 - Warnings appear if the problem is infeasible or if plotting fails.
- **Run Optimization Button**
 - Executes the solver and updates the plot and results.

Example Inputs

Example 1: Simple LP

- Objective: $\max: 2x + 3y$
- Variables: 2
- Constraints:
 - $x + y \leq 5$
 - $x \geq 1$
 - $y \geq 2$
- Variable Type: Continuous
- Method: Auto-select $\rightarrow$ Simplex

Example 2: Integer LP

- Objective: $\max: 4x + y$
- Variables: 2
- Constraints:
 - $x + y \leq 6$
 - $x \geq 2$
 - $y \geq 1$
- Variable Type: Integer
- Method: Auto-select $\rightarrow$ ILP

Example 3: Parametric LP

- Objective: $\max: x + 2y$
- Variables: 2
- Constraints:
 - $x + y \leq 4$
 - $x \geq 0$
 - $y \geq 0$
- Variable Type: Parametric
- Method: Auto-select $\rightarrow$ Parametric

Notes and Tips

- **Dikin Method** requires a strictly feasible starting point. If the default start is infeasible, it will return an error.
- **Plotting only works for 2-variable problems.** For higher dimensions, the solver runs but no plot is shown.
- **Empty or malformed constraints** will cause solver errors. Always check syntax.
- **Variable types and method selection are synchronized.** Changing one updates the other.

Output Files (Dikin Only)

When using the Dikin method:

- `dikin_path.csv` : Contains step-by-step coordinates of the optimization path
- `dikin_path.json` : Same data in JSON format

These files are saved in the working directory.

Future Enhancements (Optional Ideas)

- Add support for nonlinear objectives
- Enable 3D visualization for 3-variable problems
- Allow custom bounds on variables
- Export full solution report

8. References

1. A. Koop, H. Moock: "Lineare Optimierung - Eine anwendungsorientierte Einführung in Operations Research", 1st Ed. Spektrum (2008); G. B. Dantzig, M. N. Thalpa: "**Linear Programming 1: Introduction**", 1st Ed. Springer (1997) & "Linear Programming 2: Theory and Extensions", 1st Ed. Springer (2003); H. S. Kasana, K. D. Kumar: "Introductory Operations Research, Theory and Applications", 1st Ed. Springer (2004); D. G. Luenberger: "Linear and Nonlinear Programming", 2nd Ed. Kluwer (2004); R. J. Boucherie, A. Braaksma, H. Tijms: "Operations Research - Introduction to Models and Methods", 1st Ed. World Scientific (2022); A. J. King, S. W. Wallace: "Modeling with Stochastic Programming", 2nd Ed. Springer (2024); J. O. Royset, R. J.-B. Wets: "An Optimization Primer", 1st Ed. Springer (2021); cvxpy package: <https://www.cvxpy.org/>, <https://pypi.org/project/cvxpy/>; py-packages for operations research: <https://wiki.python.org/moin/PythonForOperationsResearch>
2. A. Meister , T. Sonar: "Numerik", 1st Ed. Springer-Spektrum (2019); S. Chapra, R. Canale: "Numerical Methods for Engineers", McGraw-Hill, 6th Edition (2010).
3. J. Kilty, A. M. McAllister: "Mathematical Modeling and Applied Calculus", 1st Ed. Oxford University Press (2018).
4. U. Kockelkorn: "Statistik für Anwender", 1st Ed. Springer (2012), s. chapters 7 - 8.
5. Robert H. Shumway, David S. Stoffer: "Time Series Analysis and Its Applications with R Examples", Springer (2011).
6. Gareth James, Daniela Witten, Trevor Hastie, Robert Tibshirani, Jonathan Taylor: "An Introduction to Statistical Learning with Applications in Python", Springer (2023).
7. Cornelis W. Oosterlee, Lech A. Grzelak: "Mathematical Modeling and Computation in Finance with Exercises and Python and MATLAB Computer Codes", World Scientific (2020).
8. Richard Szeliski: "Computer Vision - Algorithms and Applications", Springer (2022).
9. Anthony Scopatz, Kathryn D. Huff: "Effective Computation in Physics - Field Guide to Research with Python", O'Reilly Media (2015).
10. Alex Gezerlis: "Numerical Methods in Physics with Python", Cambridge University Press (2020).
11. Gary Hutson, Matt Jackson: "Graph Data Modeling in Python. A practical guide", Packt-Publishing (2023).
12. Hagen Kleinert: "Path Integrals in Quantum Mechanics, Statistics, Polymer Physics, and Financial Markets", 5th Edition, World Scientific Publishing Company (2009).
13. Peter Richmond, Jurgen Mimkes, Stefan Hutzler: "Econophysics and Physical Economics", Oxford University Press (2013).
14. A. Coryn , L. Bailer Jones: "Practical Bayesian Inference A Primer for Physical Scientists", Cambridge University Press (2017).

15. Avram Sidi: "Practical Extrapolation Methods - Theory and Applications", Cambridge university Press (2003).
16. Volker Ziemann: "Physics and Finance", Springer (2021).
17. Zhi-Hua Zhou: "Ensemble methods, foundations and algorithms", CRC Press (2012).
18. B. S. Everitt, et al.: "Cluster analysis", Wiley (2011).
19. Lior Rokach, Oded Maimon: "Data Mining With Decision Trees - Theory and Applications", World Scientific (2015).
20. Bernhard Schölkopf, Alexander J. Smola: "Learning with kernels - support vector machines, regularization, optimization and beyond", MIT Press (2009).
21. Johan A. K. Suykens: "Regularization, Optimization, Kernels, and Support Vector Machines", CRC Press (2014).
22. Sarah Depaoli: "Bayesian Structural Equation Modeling", Guilford Press (2021).
23. Rex B. Kline: "Principles and Practice of Structural Equation Modeling", Guilford Press (2023).
24. Ekaterina Kochmar: "Getting Started with Natural Language Processing", Manning (2022).
25. Rex B. Kline: "Principles and Practice of Structural Equation Modeling", Guilford Press (2023).
26. Ekaterina Kochmar: "Getting Started with Natural Language Processing", Manning (2022).
27. Jakub Langr, Vladimir Bok: "GANs in Action", Computer Vision Lead at Founders Factory (2019).
28. David Foster: "Generative Deep Learning", O'Reilly(2023).
29. Rowel Atienza: "Advanced Deep Learning with Keras: Applying GANs and other new deep learning algorithms to the real world", Packt Publishing (2018).
30. Josh Kalin: "Generative Adversarial Networks Cookbook", Packt Publishing (2018).
31. Thomas Haslwanter: „Hands-on Signal Analysis with Python: An Introduction“, Springer (2021).
32. Jose Unpingco: „Python for Signal Processing“, Springer (2023).
33. R. K. Burdick, C. M. Borror, D. C. Montgomery: "Design and Analysis of Gauge R&R Studies", 1st Ed. SIAM (2005); S. H. Derakhshan , C. V. Deutsch: "Numerical Integration of Bivariate Gaussian Distribution", Paper 405, CCG Anual Report 13 (2011).
34. C. Paar, J. Pelzl: "Understanding Cryptography", Springer (2010); H. Delfs, H. Knebl: "Introduction to Cryptography", 3rd Ed. Springer (2015); J. Katz, Y. lindell: "Introduction to Modern Cryptography", 2nd Ed, CRC Press (2015); O. Goldreich: "Foundations of Cryptography", Cambridge University Press (2008); J. P. Aumasson: "Serious Cryptography", no starch press (2018).
35. Kaggle-link: competition-documentation: <https://www.kaggle.com/competitions/drw-crypto-market-prediction>. R. Nystrom: „Game Programming Patterns“, 1st Ed. genever benning (2014); A.

A. Stepanov, D. E. Rose: „From Mathematics to Generic Programming“, 1st Ed. Addison-Wesley (2015);

36. J. Berk, P. DeMarzo: „Corporate Finance“, 6th Ed., Pearson (2023); R. W. Melicher, E. A. Norton: "Introduction to Finance", 16th Ed. WILEY (2017); Anatoly B. Schmidt: "Quantitative Finance for Physicists: An Introduction", 1st Ed. Academic Press (2005); Alex Backwell: "An Intuitive Introduction to Finance and Derivatives: Concepts, Terminology and Models", 1st Ed, Springer (2023); Michael Isichenko: "Quantitative Portfolio Management: The Art and Science of Statistical Arbitrage", 1st Ed., Springer (2021); John H. Cochrane: "Asset Pricing", Revised Ed., Princeton University Press (2005); Antti Ilmanen: "Expected Returns: An Investor's Guide to Harvesting Market Rewards", 1st Ed., WILEY (2011); Steven E. Shreve: "Stochastic Calculus for Finance I & II", 1st Ed., Springer (2004); Andrew Pole: "Statistical Arbitrage: Algorithmic Trading Insights and Techniques", 1st Ed., WILEY (2007); Mark S. Joshi: "The Concepts and Practice of Mathematical Finance", 2nd Ed., Cambridge University Press (2008);

37. E. Parzen: "Stochastic Processes", 3rd Ed. Dover Publications (2015); S. Aloorravi: "Metaprogramming with Python", 1st Ed. Packt (2022); B. Klein, P. Klein: "Funktionale Programmierung mit Python", Hanser (2025); K. Webel, D. Wied: "Stochastische Prozesse", 2. Auflage Springer (2016); L. Held: "Methoden der statistischen Inferenz", 1. Auflage Spektrum (2008); E. Cinlar: "Stochastic Processes", Dover (2013); N. Bäuerle, U. Rieder: "Finanzmathematik in diskreter Zeit", Springer-Spektrum (2017); M. Albrecht, R. Maurer: "Investment- und Risikomanagement", 3. Auflage, Schäffer Poeschel (2008); N. H. Bingham, R. Kiesel: "Risk Neutral Valuation: Pricing and Hedging of Financial Derivatives", 2. Auflage Springer (2004); T. Björk: "Arbitrage Theory in Continuous Time", 3rd Ed. Oxford University Press (2009); N. J. Cutland, A. Roux: "Derivative Pricing in Discrete Time", Springer (2013); F. Delbaen, W. Schachermayer: "The Mathematics of Arbitrage", Springer (2006); R. J. Elliott, P. E. Kopp: "Mathematics of Financial Markets", 2nd Ed. Springer (2005); H. Föllmer, A. Scheid: "A Stochastic Finance: An Introduction in Discrete Time", 3rd Ed. de Gruyter (2011); J. C. Hull: "Options, Futures and Other Derivatives", 8th Ed. Pearson (2011); J. Kremer: "Einführung in die diskrete Finanzmathematik", Springer (2005); D. Lamberton, B. Lapeyre: "Introduction to Stochastic Calculus Applied to Finance", Chapman & Hall (2007); D. G. Luenberger: "Investment Science", Oxford University Press (1998); S. R. Pliska: "Introduction to Mathematical Finance: Discrete Time Models", Blackwell (2000); A. N. Shiryaev: "Essentials of Stochastic Finance", World Scientific (2001); S. E. Shreve: "Stochastic Calculus for Finance I: The Binomial Asset Pricing Model", Springer (2004); J. Kremer: "Portfoliotheorie, Risikomanagement und die Bewertung von Derivaten", Springer (2011); L. Rüschendorf: "Mathematical Risk Analysis", Springer (2013).

38. A. Becker: "Kalman Filter - From the Ground Up", 1st Ed. private publication (2023); K. Triantafyllopoulos: "Bayesian Inference of State Space Models", 1st Ed. Springer (2021); P. Zarchan, H. Musoff: "Fundamentals of Kalman Filtering: A Practical Approach", 3rd Ed. AIAA (2009); A. Sidi: "Vector Extrapolation Methods with Applications", 1st Ed. SIAM (2019); C. Brezinski, M. R. Zaglia: "Extrapolation Methods - Theory and Practice", 2nd Ed. North-Holland (2002); C. Gardiner, P. Zoller: "Quantum Noise: A Handbook of Markovian and Non-Markovian Quantum Stochastic Methods with Applications to Quantum Optics", 3rd Ed. Springer (2004); K. Kendre: "Machine Learning for Quantum Noise Reduction", <https://arxiv.org/abs/2509.16242> (2025); D. C. Marinescu, G. M. Marinescu: "Classical and

Quantum Information", 1st Ed. Academic Press (2012); Liao, H et al.: "Machine Learning for Practical Quantum Error Mitigation", arXiv:2309.17368v2 (2024), <https://arxiv.org/pdf/2309.17368>; Streamlit: <https://streamlit.io/>; Mitiq-package: <https://quantum-journal.org/papers/q-2022-08-11-774/>, <https://arxiv.org/abs/2009.04417>; Extrapolation packages: <https://pypi.org/project/extrapolation/>